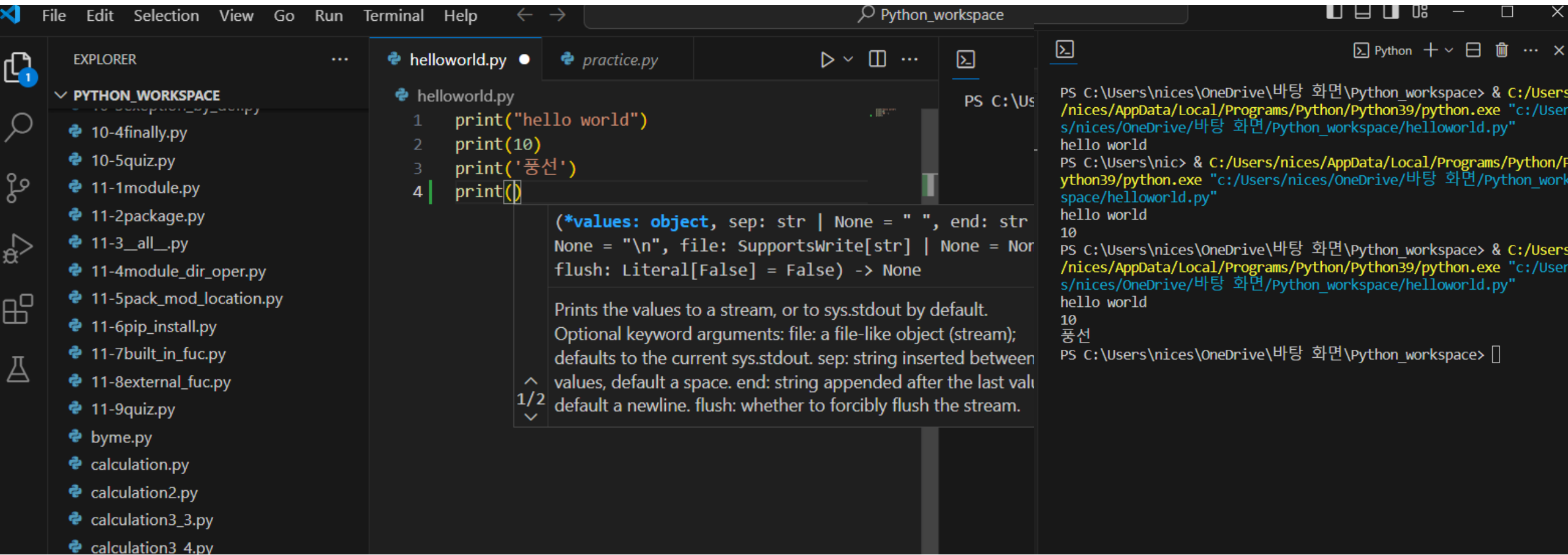


Python Basic

Practice

1. 환경 설정 & Hello World



The screenshot displays a Python IDE interface with a dark theme. On the left, the 'EXPLORER' sidebar shows a project named 'PYTHON_WORKSPACE' containing several Python files. The main editor window is open to 'helloworld.py', which contains the following code:

```
1 print("hello world")
2 print(10)
3 print('풍선')
4 print()
```

A tooltip is visible over the `print()` function call, providing documentation for the `print()` function. The tooltip text is as follows:

(*values: object, sep: str | None = " ", end: str | None = "\n", file: SupportsWrite[str] | None = None, flush: Literal[False] = False) -> None

Prints the values to a stream, or to sys.stdout by default.
Optional keyword arguments: file: a file-like object (stream); defaults to the current sys.stdout. sep: string inserted between values, default a space. end: string appended after the last value, default a newline. flush: whether to forcibly flush the stream.

On the right side of the IDE, a terminal window is open, showing the command prompt output of running the script:

```
PS C:\Users\nices\OneDrive\바탕 화면\Python_workspace> & C:/Users/nices/AppData/Local/Programs/Python/Python39/python.exe "c:/Users/nices/OneDrive/바탕 화면/Python_workspace/helloworld.py"
hello world
PS C:\Users\nic> & C:/Users/nices/AppData/Local/Programs/Python/Python39/python.exe "c:/Users/nices/OneDrive/바탕 화면/Python_workspace/helloworld.py"
hello world
10
풍선
PS C:\Users\nices\OneDrive\바탕 화면\Python_workspace>
```

2-1. 숫자 자료형 2-2. 문자열 자료형 2-3. boolean 자료형, 2-4. 변수, 2-5. 주석

```
helloworld.py M × calculation.py
helloworld.py
1 print("hello world")
2 print(10)
3 print('풍선')
4 | print(3>5)
```

```
practice.py × calculation.py
practice.py > ...
1 # 애완동물을 소개해 주세요
2 animal = "고양이"
3 name = "해피"
4 age = 4
5 hobby = "낮잠"
6 is_adult = age >= 3
7
8 print("우리집 " + animal + "의 이름은 " + name + " 애요")
9 print(name + "는" + str(age) + "살이며," + hobby + "을 아주 좋아해요")
10 print(name + "는 어른일까요?" + str(is_adult))
11
12
```

3-1.연산자 3-2.간단한수식 3-3.숫자처리함수

```
calculation.py
1  print(1+1)
2  print(3-2)
3  print(5*2)
4  print(6/3)
5
6  print(2**3) # **는 지수를 뜻함 (2의 3승)
7  print(5%3) # %는 5를 3으로 나눈 나머지
8  print(5//3) # //는 5를 3으로 나눈 몫
9
10 print(10 > 3)
11 print(4 >= 7)
12 print(10 < 3)
13 print(5 <= 3)
14
15 print(3 == 3) # == 는 똑같은지 확인하는 연산
16 print(4 == 2)
17 print(3 + 4 == 7)
18
19 print(1 != 3) # !=는 같지 않다는 뜻 (not과 같은 의미) True로 나옴
20 print(not (1 != 3)) # 같지 않다는 것이 not 이다 --> False로 나옴
21
22 print ((3 > 0) and (3 < 5)) # True
23 print ((3 > 0) & (3 < 5))
24
25 print ((3 > 0) or (3 > 5)) # or 이므로 둘중하나 True이므로 True
26
27 print (3 < 5 < 7)
28 print (3 < 5 < 4)
```

4-1. 문자열 (string) & 문자열처리

```
4-1string.py U X
4-1string.py > ...
1 sentence = '나는 소년입니다'
2 print(sentence)
3
4
5 sentence2 = """
6 나는 소년이고,
7 파이선은 쉬워요
8 """
9 print(sentence2)
```

```
helloworld.py M 4-3string_func.py U X calculation.py
4-3string_func.py > ...
1 python = "Python is Amazing"
2
3 print(python.lower())
4 print(python.upper())
5 print(python[0].isupper())
6 #python의 0번째 (첫번째) 글자가 upper인가?
7 print(len(python))
8 print(python.replace("Python", "Sang Wook"))
9
10 index = python.index("n")
11 print(index)
12 index = python.index("n", index + 1)
13 print(index)
14
15 print(python.find("n"))
16 print(python.find("Java"))
17 # python에 원하는 값(내용이 없으면 (않찾아지면) -1을 표시
18 print(python.index("Java"))
19 # python에 원하는 값(내용)이 없으면 오류가 나면서
20 # 더이상 실행을 하지 않고 종료됨
21
22 # 위에서 종료되었으므로 그 아래 명령들은 실행 안됨
23
24 print(python.count("n"))
25 # python에 n이 총 몇번 나오는지 세기
```

5-1.리스트 5-2.사전 5-3.튜플

```
helloworld.py M 5-1list.py X calculation.py
5-1list.py > ...
1  # 리스트
2
3  # 지하철 칸별로 10명, 20명, 30명
4  #subway1 = 10
5  #subway2 = 20
6  #subway3 = 30
7
8  # subway = [10, 20, 30]
9  # print (subway)
10
11 # subway = ["유재석", "조세호", "박명수"]
12 # print (subway)
13
14 # # 조세호가 몇 번째 칸에 타고 있는가?
15 # print(subway.index("조세호")) # index는 0, 1, 2 로 시작됨
16
17 # # 하하가 다음 정류장에서 다음 칸에 탑
18 # subway.append("하하")
19 # print(subway)
20
21 # # 정형돈을 유재석 / 조세호 사이에 태워봄
22 # subway.insert(1, "정형돈")
23 # print(subway)
```

```
helloworld.py M 5-2dictionary.py X calculation.py
5-2dictionary.py > ...
1  # cabinet = {3:"유재석", 100:"김태호"}
2  # print(cabinet[3])
3  # print(cabinet[100])
4
5  # print(cabinet.get(3))
6
7  # print(cabinet[5])
8  # print("hi")
9
10
11
12
13
14
15
16
17
18
19
20 cabinet = {"A-3":"유재석", "B-100":"김태호"}
21 print(cabinet["A-3"])
22 print(cabinet["B-100"])
23 # 새손님
24 print(cabinet)
25 cabinet["A-3"] = "김종국" #유재석이 빠지고 김종국이 들어감
26 cabinet["C-20"] = "조세호"
27 print(cabinet)
28
```

5-4. 세트 5-5. 자료구조의 변경

```
helloworld.py M 5-4set.py X calculation.py
5-4set.py > ...
1 # 집합 set
2 # 중복 안됨, 순서 없음
3 my_set = {1,2,3,3,3}
4 print(my_set)
5
6 java = {"유재석", "김태호", "양세형"}
7 python = set(["유재석", "박명수"])
8
9 # 교집합 (java와 python을 모두 할 수 있는 개발자)
10 print (java & python)
11 print (java.intersection(python))
12
13 # 합집합 (java를 할 수 있거나 python을 할 수 있는 개발자)
14 print (java | python)
15 print (java.union(python))
16
17 # 차집합 (java 할수 있지만 python은 할줄 모르는 개발자)
18 print (java - python)
19 print (java.difference(python))
20
21 # 교육받아서 python 할 줄 아는 사람이 늘어남
22 python.add("김태호")
23 print (python)
24
25 # 자바를 까먹은 사람
26 java.remove("김태호")
27 print (java)
```

```
helloworld.py M 5-5struc_change.py X calculation.py
5-5struc_change.py > ...
1 # 자료구조의 변경
2 # 커피숍
3 menu = {"커피", "우유", "주스"}
4 print(menu, type(menu))
5
6 menu = list(menu)
7 print(menu, type(menu))
8
9 menu = tuple(menu)
10 print(menu, type(menu))
11
12 menu = set(menu)
13 print(menu, type(menu))
14
15
```

```
PS C:\Users\nices\OneDrive\바탕 화면\Python_work
space> & C:/Users/nices/AppData/Local/Programs/P
ython/Python39/python.exe "c:/Users/nices/OneDri
ve/바탕 화면/Python_workspace/5-5struc_change.py
"

{'우유', '커피', '주스'} <class 'set'>
['우유', '커피', '주스'] <class 'list'>
('우유', '커피', '주스') <class 'tuple'>
{'우유', '커피', '주스'} <class 'set'>
PS C:\Users\nices\OneDrive\바탕 화면\Python_work
space>
```

6-1.if (분기 or 선택) 6-2.for (반복) 6-3.while 6-4.continue 와 break 6-5.한 줄 for

```
helloworld.py M 6-1if.py × calculation.py
6-1if.py > ...
1  # weather = input("오늘 날씨는 어때요?")
2  # # if 조건:
3  # #     실행 명령문
4  # if weather == "비" or weather == "눈":
5  #     print("우산을 챙기세요")
6  # elif weather == "미세먼지":
7  #     print("마스크를 챙기세요")
8  # else:
9  #     print("준비물 필요 없어요")
10
11 temp = int(input("기온은 어때요?"))
12 if 30 <= temp:
13     print("너무 더워요. 나가지 마세요")
14 elif 10 <= temp and temp < 30:
15     print("괜찮은 날씨예요")
16 elif 0 <= temp < 10:
17     print("외투를 챙기세요")
18 else:
19     print("너무 추워요. 나가지 마세요")
20
```

```
6-2for.py > ...
1  # print("대기번호 : 1")
2  # print("대기번호 : 2")
3  # print("대기번호 : 3")
4  # print("대기번호 : 4")
5
6  # for waiting_no in [1, 2, 3, 4]:
7  #     print("대기번호 : {}".format(waiting_no))
8
9  # for waiting_no in range(5): # 0,1,2,3,4
10 #     print("대기번호 : {}".format(waiting_no))
11
12 for waiting_no in range(1, 6): # 1부터 6미만까지
13     print("대기번호 : {}".format(waiting_no))
14
15 starbucks = ["아이언맨", "토르", "아이엠 그루트"]
16 for customer in starbucks: #스타벅스의 손님을 한명씩 부르는 것
17     print("{} , 커피가 준비되었습니다.".format(customer))
18
```

6-3.while (조건 반복), continue & break

```
6-3while.py > ...
1  # while 손님 5번 불러서 안오면 커피를 버린다
2  # customer = "토르"
3  # index = 5
4  # while index >= 1:
5  #     print("{0}, 커피가 준비되었습니다. {1} 번 남았어요.".format(customer, index))
6  #     index -= 1
7  #     if index == 0:
8  #         print("커피는 폐기처분 되었습니다")
9
10 # customer = "아이언맨"
11 # index = 1
12 # while True:
13 #     print("{0}, 커피가 준비 되었습니다. 호출 {1} 회".format(customer, index))
14 #     index += 1
15
16
17 customer = "토르"
18 person = "unknown"
19
20 while person !=customer : # person 이 customer와 같을때까지 계속 반복
21     print("{0}, 커피가 준비되었습니다".format(customer))
22     person = input("이름이 어떻게 되세요?")
23
24
```

7-1. 함수

7-2. 전달값과 반환값

```
7-1function.py > ...
1  def open_account():
2      print("새로운 계좌가 생성되었습니다")
3
4  # open_account()
5
6  def deposit(balance, money): # 입금
7      print("입금이 완료되었습니다. 잔액은 {0} 원입니다".format(balance + money))
8      return balance + money
9
10 def withdraw(balance, money): # 출금
11     if balance >= money: # 잔액이 출금보다 많으면
12         print("출금이 완료되었습니다. 잔액은 {0} 원입니다".format(balance - money))
13         return balance - money
14     else:
15         print("출금이 완료되지 않았습니다. 잔액은 {0} 원입니다".format(balance))
16         return balance
17
18 def withdraw_night(balance, money): # 저녁에 출금
19     commission = 100 # 수수료 100원
20     if balance >= money + commission: # 잔액이 출금 + 수수료보다 많으면
21         print("출금이 완료되었습니다. 수수료 {0} 원이며, 잔액은 {1} 원입니다".format(commission, balance - money - commission))
22         return commission, balance - money - commission
23     else:
24         print("출금이 완료되지 않았습니다. 잔액은 {0} 원입니다".format(balance))
25         return balance
26
27
28
29
30 balance = 0 #잔액
31 balance = deposit(balance, 1000)
32 # balance = withdraw(balance, 2000)
33 # balance = withdraw(balance, 500)
34 balance = withdraw_night(balance, 500)
```

기본값, 키워드값 7-5.가변인자

7-5variable.py > ...

```
1  # def profile(name, age, lang1, lang2, lang3, lang4, lang5):
2  #     print("이름 : {0}\t나이 : {1}\t".format(name, age), end=" ")
3  #     print(lang1, lang2, lang3, lang4, lang5)
4
5  # profile("유재석", 20, "python", "Java", "C", "C++", "C#")
6  # profile("김태호", 25, "Kotlin", "Swift", "", "", "")
7
8  def profile(name, age, *language):
9      print("이름 : {0}\t나이 : {1}\t".format(name, age), end=" ")
10     for lang in language:
11         print(lang, end=" ")
12     print()
13
14
15     profile("유재석", 20, "python", "Java", "C", "C++", "C#", "Javascript")
16     profile("김태호", 25, "Kotlin", "Swift")
17
```

7-6.지역변수와 전역변수

7-6local_var.py > ...

```
1  # gun = 10
2
3  # def checkpoint(soldiers): # 경계근무
4  #     gun = 20
5  #     gun = gun -soldiers
6  #     print("[함수 내] 남은 총 : {0}".format(gun))
7
8  # print("전체 총 : {0}".format(gun))
9  # checkpoint(2) # 2 명이 경계 근무 나감
10 # print("남은 총 : {0}".format(gun))
11
12 gun = 10
13
14 def checkpoint(soldiers): # 경계근무
15     global gun # 전역 공간에 있는 gun 사용
16     gun = gun -soldiers
17     print("[함수 내] 남은 총 : {0}".format(gun))
18
19 def checkpoint_ret(gun, soldiers):
20     gun = gun - soldiers
21     print("[함수 내] 남은 총: {0}".format(gun))
22     return gun
23
24
25 print("전체 총 : {0}".format(gun))
26 gun = checkpoint_ret (gun, 2) # 2 명이 경계 근무 나감
27 print("남은 총 : {0}".format(gun))
28
```

8-1.표준입출력, 출력포맷 8-3.파일입출력

8-1in_output.py > ...

```
1  # print("Phthon", "Java", sep=" , ", end="?")
2  # print("무엇이 더 재미있을까요?")
3
4  # import sys
5  # print("Phthon", "Java", file=sys.stdout)
6  # print("Phthon", "Java", file=sys.stderr)
7
8
9  # 시험 성적
10 # scores = {"수학":0, "영어":50, "코딩":100}
11 # for subject, score in scores.items():
12 #     #print(subject, score)
13 #     print(subject.ljust(8), str(score).rjust(4), sep=":")
14
15
16 # 은행 대기순번표
17 # 001, 002, 003, ...
18 # for num in range(1, 21):
19 #     print("대기번호 : " + str(num).zfill(3))
20
21
22 #answer = input("아무값이나 입력하세요 : ")
23 answer = 10
24 print(type(answer))
25 #print("입력하신 값은 " + answer + "입니다")
```

8-3file_in_output.py > ...

```
1  # score_file = open("score.txt", "w", encoding="utf8")
2  # print("수학 : 0", file=score_file)
3  # print("영어 : 50", file=score_file)
4  # score_file.close()
5
6  # score_file = open("score.txt", "a", encoding="utf8")
7  # score_file.write("과학 : 80")
8  # score_file.write("\n코딩 : 100")
9  # score_file.close()
10
11 # score_file = open("score.txt", "r", encoding="utf8")
12 # print(score_file.read())
13 # score_file.close()
14
15 # score_file = open("score.txt", "r", encoding="utf8")
16 # print(score_file.readline(), end="") # 줄별로 읽기, 한
17 # print(score_file.readline(), end="")
18 # print(score_file.readline())
19 # print(score_file.readline())
20 # score_file.close()
21
22 # score_file = open("score.txt", "r", encoding="utf8")
23 # while True:
24 #     line = score_file.readline()
25 #     if not line:
26 #         break
27 #     print(line, end="")
28 # score_file.close()
29
30 score_file = open("score.txt", "r", encoding="utf8")
31 lines = score_file.readlines() # list 형태로 저장
32 for line in lines:
33     print(line)
34 score_file.close()
```

8-4.pickle 8-5.with

8-4pickle.py > ...

```
1  import pickle
2  # profile_file = open("profile.pickle", "wb")
3  # profile = {"이름": "박명수", "나이":30, "취미":["축구", "골프", "코딩"]}
4  # print(profile)
5  # pickle.dump(profile, profile_file) #profile에 있는 정보를 file에 저장
6  # profile_file.close()
7
8  profile_file = open("profile.pickle", "rb")
9  profile = pickle.load(profile_file)
10 print(profile)
11 profile_file.close()
```

8-5with.py > ...

```
1  # import pickle
2  # with open("profile.pickle", "rb") as profile_file:
3  #     print(pickle.load(profile_file))
4
5  # with open("study.txt", "w", encoding="utf8") as study_file:
6  #     study_file.write("파이썬을 열심히 공부하고 있어요")
7
8
9  with open("study.txt", "r", encoding="utf8") as study_file:
10 |     print(study_file.read())
```

9-1.클래스 9-2._init_ 9-3.멤버변수

C: > nicesw > lecture > 2024 > 2학기 > 전산물리학 > lecture 2 > class_student_info.py > ...

```
1 class Student:
2     def __init__(self, name, age, location):
3         self.name = name
4         self.age = age
5         self.location = location
6
7     def __str__(self):
8         return f"Name: {self.name}, Age: {self.age}, Location: {self.location}"
9
10 class Classroom:
11     def __init__(self, *students_info):
12         self.students = []
13         for info in students_info:
14             self.add_student(*info)
15
16     def add_student(self, name, age, location):
17         student = Student(name, age, location)
18         self.students.append(student)
19
20     def list_students(self):
21         for student in self.students:
22             print(student)
23
24     # 여러 학생 정보를 넣어 Classroom 인스턴스를 생성합니다.
25     classroom = Classroom(('이상욱', 47, '서울'), ('홍길동', 16, '부산'), ('김철수', 15, '대구'))
26     # 학생들의 목록을 출력합니다.
27     classroom.list_students()
28
29     # # 두 번째 Classroom 인스턴스를 생성합니다.
30     # classroom2 = Classroom(('박영희', 17, '인천'), ('최민수', 16, '광주'), ('이민호', 18, '대전'))
31
32     # # classroom2에 있는 학생들의 목록을 출력합니다.
33     # classroom2.list_students()
```

9-4.메소드 9-5.상속

```
81 # 부모 클래스
82 class Animal:
83     def __init__(self, name, sound):
84         self.name = name
85         self.sound = sound
86
87     def make_sound(self):
88         print(f"{self.name} makes a {self.sound} sound.")
89
90 # 자식 클래스 (Animal 클래스 상속)
91 class Dog(Animal):
92     def __init__(self, name, sound, breed):
93         Animal.__init__(self, name, sound) # 부모 클래스의 __init__ 직접 호출
94         self.breed = breed
95
96     def info(self):
97         print(f"{self.name} is a {self.breed} and it barks.")
98
99 # 자식 클래스 (Animal 클래스 상속)
100 class Cat(Animal):
101     def __init__(self, name, sound, color):
102         Animal.__init__(self, name, sound) # 부모 클래스의 __init__ 직접 호출
103         self.color = color
104
105     def info(self):
106         print(f"{self.name} is a {self.color} cat and it meows.")
107
108 # 객체 생성
109 dog = Dog("Buddy", "bark", "Golden Retriever")
110 cat = Cat("Whiskers", "meow", "black")
111
112 # 메서드 호출
113 dog.make_sound() # Buddy makes a bark sound.
114 dog.info()       # Buddy is a Golden Retriever and it barks.
115
116 cat.make_sound() # Whiskers makes a meow sound.
117 cat.info()       # Whiskers is a black cat and it meows.
```

10-1.예외처리

try-except 문

10-1exception.py > ...

```
1  try:
2      print("나누기 전용 계산기입니다.")
3      num1 = int(input("첫번째 숫자를 입력하세요 : "))
4      num2 = int(input("두번째 숫자를 입력하세요 : "))
5      print("{0} / {1} = {2}".format(num1, num2, int(num1/num2)))
6  except ValueError:
7      print("에러! 잘못된 값을 입력하였습니다.")
8  except ZeroDivisionError as err:
9      print(err)
10 except Exception as err:
11     print("알 수 없는 에러가 발생하였습니다.")
12     print(err)
```

Q & A